

Day 1: Single Artificial Neuron using ReLU

```
import numpy as np

X = np.array([0.8, 0.4, 1.0])

W = np.array([0.5, -0.3, 0.9])

b = 0.1

z = np.dot(W, X) + b

print("Input Vector:", X)
print("Weight Vector:", W)
print("Bias:", b)
print("Weighted Sum (z):", round(z, 4))

def relu(z):
    return max(0, z)

output = relu(z)

print("Neuron Output after ReLU:", round(output, 4))

Input Vector: [0.8 0.4 1. ]
Weight Vector: [ 0.5 -0.3  0.9]
Bias: 0.1
Weighted Sum (z): 1.28
Neuron Output after ReLU: 1.28
```

Day 2: Comparison of Activation Functions

```
import numpy as np

def relu(z):
    return max(0, z)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def tanh_fn(z):
```

```

    return np.tanh(z)

test_values = [-3, -1, 0, 1, 3]

print(f"{'Input(z)':<10}{'ReLU':<10}{'Sigmoid':<12}{'Tanh':<12}")
print("-" * 45)

for z in test_values:
    print(f"{'z':<10}{'relu(z)':<10.4f}{'sigmoid(z)':<12.4f}{'tanh_fn(z)':<12.4f}")

```

Input(z)	ReLU	Sigmoid	Tanh
-3	0.0000	0.0474	-0.9951
-1	0.0000	0.2689	-0.7616
0	0.0000	0.5000	0.0000
1	1.0000	0.7311	0.7616
3	3.0000	0.9526	0.9951

Bonus (Using Graph)

```

import numpy as np
import matplotlib.pyplot as plt

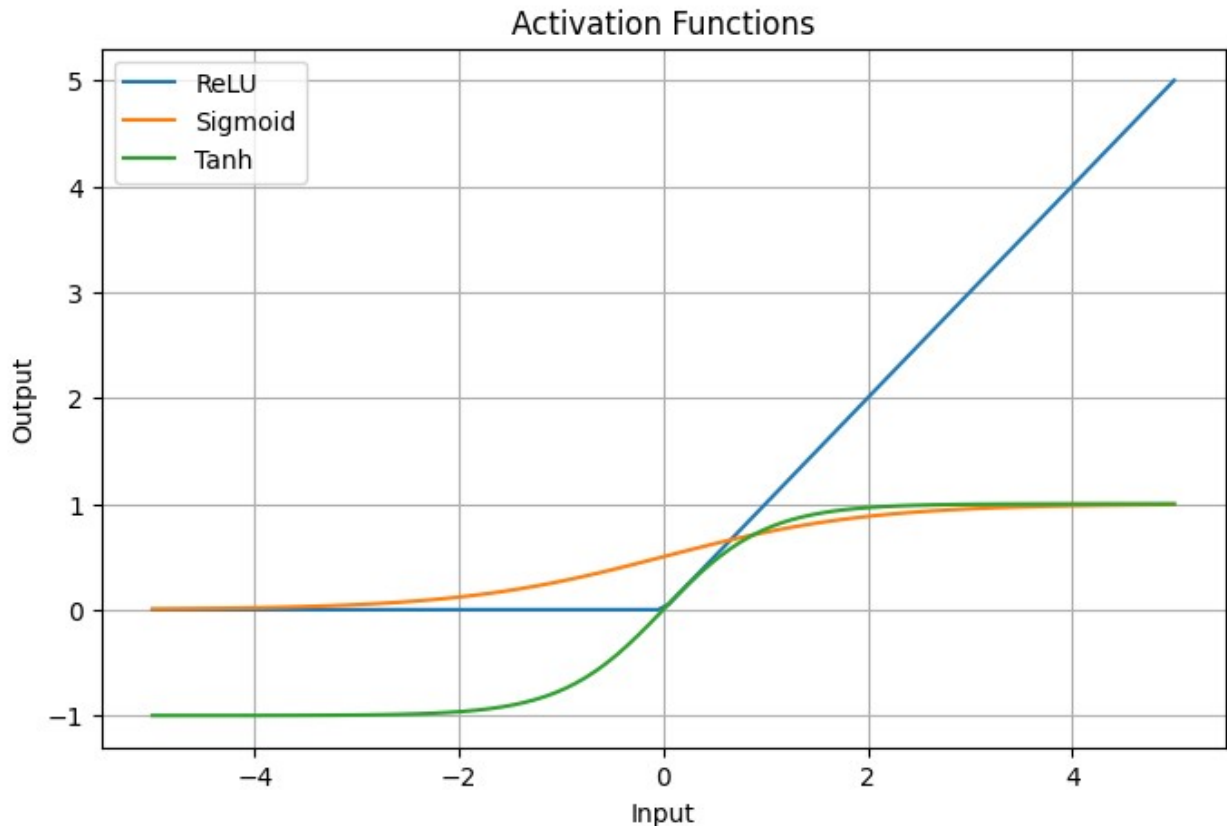
x = np.linspace(-5, 5, 100)

relu = np.maximum(0, x)
sigmoid = 1/(1 + np.exp(-x))
tanh = np.tanh(x)

plt.figure(figsize=(8,5))
plt.plot(x, relu, label='ReLU')
plt.plot(x, sigmoid, label='Sigmoid')
plt.plot(x, tanh, label='Tanh')

plt.title("Activation Functions")
plt.xlabel("Input")
plt.ylabel("Output")
plt.legend()
plt.grid(True)
plt.show()

```



Part A: Hugging Face Integration

```
!pip install transformers datasets -q
from huggingface_hub import login
login()

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(

{"model_id": "04de20c9124644c4be27bef868a7cdd4", "version_major": 2, "version_minor": 0}

from transformers import pipeline
```

```
classifier = pipeline('sentiment-analysis')

result = classifier('I absolutely love this generative AI course!')
print(result)
```

```
generator = pipeline('text-generation', model='gpt2')
```

```
output = generator(
    'Generative AI is',
    max_new_tokens=50,
    num_return_sequences=2
)
```

```
print(output[0]['generated_text'])
print(output[1]['generated_text'])
```

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.

```
{"model_id": "2473c337bd9b4570a2b552e69e2b8262", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "00c8cc86ff3144fa99fbdce93a9df5f3", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "29eeaaa058544e80aa71f82f15b19179", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "cf8eb1f92dd347c28cea735cacbda112", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2fd6fee4a90f4bf4b947940f7e3cf30d", "version_major": 2, "version_minor": 0}
```

```
[{'label': 'POSITIVE', 'score': 0.9998745918273926}]
```

```
{"model_id": "f89f35eba23d4f84bdb53990baa1c81c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1f76ecae7b5546a1a96128e0c9961a5b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e3b0936c7e434949926fb960139ca925", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "503c80ebbd1b476492943aa70e7ea152", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "47a134791c45452aa0c58b17358d9278", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5b23b0adfec84610ab3b2841db9f205e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "a8db07cc93314ed0b48a5327d72cea64", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "48e865b502f141beb5ff2eb527a3cd50", "version_major": 2, "version_minor": 0}
```

[transformers] Passing `generation_config` together with generation-related arguments (`{'num_return_sequences', 'max_new_tokens'}`) is deprecated and will be removed in future versions. Please pass either a `generation_config` object OR all generation parameters explicitly, but not both.

[transformers] Setting `pad_token_id` to `eos_token_id`: 50256 for open-end generation.

[transformers] Both `max_new_tokens` (=50) and `max_length` (=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information.

(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] Ignoring `clean_up_tokenization_spaces=True` for BPE tokenizer GPT2Tokenizer. The `clean_up_tokenization` post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set `clean_up_tokenization_spaces=False` to suppress this warning, or set `clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True` to force cleanup anyway.

Generative AI is a simple way to work with your existing data. This is in no way an ideal way of doing things. We are constantly improving our hardware and software.

To get started with the demo, download the release from this repo.

It's a

Generative AI is a way to create an AI that works with the current state of the world. It allows the user to build a system that is based on a specific set of properties, but can also work on a variety of different worlds.

It is not

```
!pip install bertviz transformers -q
```

0.0/157.5 kB ? eta -:-:-
157.5/157.5 kB 5.5 MB/s eta
0:00:00
140.5/140.5 kB 12.7 MB/s eta

```
0:00:00
15.2/15.2 MB 39.3 MB/s eta
0:00:00
4.9/4.9 MB 77.2 MB/s eta
0:00:00
88.6/88.6 kB 6.9 MB/s eta
0:00:00
```

```
from bertviz import head_view
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained(
    'bert-base-uncased'
)

model = BertModel.from_pretrained(
    'bert-base-uncased',
    output_attentions=True
)

sentence = "The animal didn't cross the street because it was too tired"

inputs = tokenizer(
    sentence,
    return_tensors='pt'
)

with torch.no_grad():
    outputs = model(**inputs)

attention = outputs.attentions

tokens = tokenizer.convert_ids_to_tokens(
    inputs['input_ids'][0]
)

head_view(attention, tokens)

{"model_id": "809d72184e32454abf596f3e1aca0333", "version_major": 2, "version_minor": 0}

{"model_id": "65318516a651412199bd9e0ab0f8b67e", "version_major": 2, "version_minor": 0}

{"model_id": "e6ab7f33fdd44b8ca433be1887ca34b2", "version_major": 2, "version_minor": 0}

{"model_id": "8858a33f393a48b9b975dc784ac8528c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "526e6aa67e25488886e79a54902c650c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "eb12b4392c9549259ea0dd1cb21645bd", "version_major": 2, "version_minor": 0}
```

```
[transformers] BertModel LOAD REPORT from: bert-base-uncased
```

Key	Status		
cls.seq_relationship.weight	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.Javascript object>

Day 3: Tokenization, NLP Operations, Text Generation, and IBM Granite Integration

```
import tiktoken

enc = tiktoken.encoding_for_model('gpt-4')

tokens = enc.encode(
    'Generative AI is transforming the world!'
)

print(f'Tokens: {tokens}')
print(f'Token count: {len(tokens)}')
print(f'Decoded: {enc.decode(tokens)}')

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    'bert-base-uncased'
)

encoded = tokenizer(
    'Hello, I am learning about LLMs!',
    return_tensors='pt'
)
```

```

print('Input IDs:', encoded['input_ids'])
print(
    'Tokens:',
    tokenizer.convert_ids_to_tokens(
        encoded['input_ids'][0]
    )
)

```

Tokens: [5648, 1413, 15592, 374, 46890, 279, 1917, 0]

Token count: 8

Decoded: Generative AI is transforming the world!

Input IDs: tensor([[101, 7592, 1010, 1045, 2572, 4083, 2055, 2222, 5244, 999, 102]])

Tokens: ['[CLS]', 'hello', ',', 'i', 'am', 'learning', 'about', 'll', '##ms', '!', '[SEP]']

```
import nltk
```

```

nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger_eng')

```

```

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag

```

```
text = "The boys are running quickly to the playground."
```

```

tokens = word_tokenize(text)
print("Tokens:", tokens)

```

```

stop_words = set(stopwords.words('english'))
filtered = [

```

```

    word for word in tokens
    if word.lower() not in stop_words
]
print("After Stopword Removal:", filtered)

```

```

stemmer = PorterStemmer()
stems = [stemmer.stem(word) for word in filtered]
print("Stemmed:", stems)

```

```

lemmatizer = WordNetLemmatizer()
lemmas = [
    lemmatizer.lemmatize(word)
    for word in filtered

```

```

]
print("Lemmatized:", lemmas)

tags = pos_tag(tokens)
print("POS Tags:", tags)

[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.

Tokens: ['The', 'boys', 'are', 'running', 'quickly', 'to', 'the',
'playground', '.']
After Stopword Removal: ['boys', 'running', 'quickly', 'playground',
'.']
Stemmed: ['boy', 'run', 'quickli', 'playground', '.']
Lemmatized: ['boy', 'running', 'quickly', 'playground', '.']
POS Tags: [('The', 'DT'), ('boys', 'NNS'), ('are', 'VBP'), ('running',
'VBG'), ('quickly', 'RB'), ('to', 'TO'), ('the', 'DT'), ('playground',
'NN'), ('.', '.')]

from transformers import pipeline

generator = pipeline(
    'text-generation',
    model='gpt2',
    device=-1
)

prompt = (
    'Generative AI will transform education because'
)

outputs = generator(
    prompt,
    max_new_tokens=100,
    num_return_sequences=3,
    temperature=0.8,
    do_sample=True,
    top_p=0.95,
    top_k=50
)

for i, output in enumerate(outputs):
    print(f'\n--- Option {i+1} ---')
    print(output['generated_text'])

```

```
{"model_id": "da02eb6ce3554bfd880cbce79bd294cc", "version_major": 2, "version_minor": 0}
```

[transformers] Passing ``generation_config`` together with generation-related arguments (`{'top_p', 'num_return_sequences', 'max_new_tokens', 'top_k', 'do_sample', 'temperature'}`) is deprecated and will be removed in future versions. Please pass either a ``generation_config`` object OR all generation parameters explicitly, but not both.

[transformers] Setting ``pad_token_id`` to ``eos_token_id``:50256 for open-end generation.

[transformers] Both ``max_new_tokens`` (=100) and ``max_length`` (=50) seem to have been set. ``max_new_tokens`` will take precedence. Please refer to the documentation for more information.

(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

--- Option 1 ---

Generative AI will transform education because it's a way to connect a person's brain to the world around them.

"I don't want to believe that people have this technology to do anything but teach and educate themselves," said Richard Smith, a professor of psychology at the University of California, Santa Barbara.

As a result, he said, "we can't fully understand how people learn to use the same kind of technology to learn their own lives."

Smith believes that people are simply making more and more conscious

--- Option 2 ---

Generative AI will transform education because of the potential of its potential to disrupt the private sector and its ability to influence the outcome of elections.

The technology has been described as a "superb machine", and the new software has been described as a "supercomputer".

The new AI will transform education because of the potential of its potential to disrupt the private sector and its ability to influence the outcome of elections.

The technology has been described as a "superb machine", and the new software has been described as

--- Option 3 ---

Generative AI will transform education because it will be better at predicting future outcomes rather than relying on existing knowledge or techniques to learn from past events.

In a recent paper, the University of Cambridge's Department of Physics and Engineering, lead author David Haines, suggested that an AI could also learn from past experiences, such as people or things they have seen, and the future they may find themselves in. Haines' paper, "A Novel Autonomous Artificial Intelligence: A Method for Predicting Future Events," was published

```
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer
)
import torch

model_id = (
    'ibm-granite/granite-3.1-2b-instruct'
)

tokenizer = AutoTokenizer.from_pretrained(
    model_id
)

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
    device_map='auto'
)

messages = [
    {
        'role': 'user',
        'content':
            'Explain what a transformer is in 3 bullet points for a
student.'
    }
]

input_ids = tokenizer.apply_chat_template(
    messages,
    return_tensors='pt'
)['input_ids'].to(model.device)

output = model.generate(
    input_ids,
    max_new_tokens=256,
    temperature=0.7,
    do_sample=True
)

response = tokenizer.decode(
    output[0][input_ids.shape[1]:],
```

```

    skip_special_tokens=True
)

print(response)

{"model_id": "266b560599cd43638250e56b8bce3b07", "version_major": 2, "version_minor": 0}

{"model_id": "c48897e1bb774c3b9c8afb5a5ce11623", "version_major": 2, "version_minor": 0}

{"model_id": "527d5c03441b423e8208991229e7b2e9", "version_major": 2, "version_minor": 0}

{"model_id": "61d446179844401baca72c357c60605c", "version_major": 2, "version_minor": 0}

{"model_id": "be1d6291f4ce417a9433e74f9cde4403", "version_major": 2, "version_minor": 0}

[transformers] `torch_dtype` is deprecated! Use `dtype` instead!

{"model_id": "b586e30c087344e689b0b15652463d2d", "version_major": 2, "version_minor": 0}

{"model_id": "c5f89d5b5588480c94f3e4076d52cb59", "version_major": 2, "version_minor": 0}

{"model_id": "15f36c7f7eae4fcdbb2531278549d302", "version_major": 2, "version_minor": 0}

{"model_id": "b9c1fad45d874d97ba93e1081613368b", "version_major": 2, "version_minor": 0}

{"model_id": "2ee9f7980d31452f84fa63ead73e9b23", "version_major": 2, "version_minor": 0}

[transformers] The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your input's `attention_mask` to obtain reliable results.
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer TokenizersBackend. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force cleanup anyway.

- A transformer is a type of deep learning model architecture, primarily used in natural language processing (NLP) tasks, introduced

```

in the groundbreaking 2017 paper "Attention is All You Need" by Vaswani et al. It's designed to handle sequential data, like sentences or documents, and convert them into meaningful representations (known as embeddings) that capture contextual relationships among words.

- Unlike traditional recurrent neural networks (RNNs), transformers don't employ a sequential processing mechanism. Instead, they utilize self-attention mechanisms, also called scaled dot-product attention, which allow the model to focus on various parts of the input sequence simultaneously, thereby reducing the computational complexity and preserving more information during the transformation.

- In the context of NLP, transformers have significantly improved the performance of tasks such as machine translation, text summarization, sentiment analysis, and question-answering. These models can efficiently process long sequences, making them ideal for understanding and generating human language with greater accuracy and fluency.

Day 4: Exploring Hugging Face Hub, Bias Detection, and AI Image Generation

```
!pip install transformers datasets accelerate
!pip install diffusers
!pip install --upgrade huggingface_hub

from huggingface_hub import login

login(token="your_token")

Requirement already satisfied: transformers in
/usr/local/lib/python3.12/dist-packages (5.10.1)
Requirement already satisfied: datasets in
/usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: accelerate in
/usr/local/lib/python3.12/dist-packages (1.13.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (1.19.0)
Requirement already satisfied: numpy>=1.17 in
/usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (26.2)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in
/usr/local/lib/python3.12/dist-packages (from transformers)
(2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.25.1)
```

Requirement already satisfied: safetensors>=0.4.3 in
/usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in
/usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from datasets) (3.29.1)
Requirement already satisfied: pyarrow>=15.0.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in
/usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: xxhash in
/usr/local/lib/python3.12/dist-packages (from datasets) (3.7.0)
Requirement already satisfied: multiprocessing<0.70.17 in
/usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2025.3.0)
Requirement already satisfied: psutil in
/usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: torch>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from accelerate) (2.11.0+cpu)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (3.14.0)
Requirement already satisfied: click>=8.4.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.5.0->transformers) (8.4.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.5.0->transformers) (1.5.1)
Requirement already satisfied: httpx<1,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.5.0->transformers) (0.28.1)
Requirement already satisfied: typing-extensions>=4.1.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=1.5.0->transformers) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
>datasets) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2026.5.20)

Requirement already satisfied: setuptools<82 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.6.1)

Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.1.6)

Requirement already satisfied: shellingham>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer->transformers) (1.5.4)

Requirement already satisfied: rich>=13.8.0 in
/usr/local/lib/python3.12/dist-packages (from typer->transformers) (13.9.4)

Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from typer->transformers) (0.0.4)

Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.2)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (2.6.2)

Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.4.0)

Requirement already satisfied: attrs>=17.3.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (26.1.0)

Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.8.0)

Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,! =4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (6.7.1)

Requirement already satisfied: propcache>=0.2.0 in

```
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (0.5.2)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets) (1.24.2)
Requirement already satisfied: anyio in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<2.0,>=1.5.0->transformers) (4.13.0)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<2.0,>=1.5.0->transformers) (1.0.9)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.12/dist-packages (from httpcore==1.*-
>httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (0.16.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas->datasets) (1.17.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer-
>transformers) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer-
>transformers) (2.20.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3-
>torch>=2.0.0->accelerate) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.0.0-
>accelerate) (3.0.3)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich>=13.8.0->typer->transformers) (0.1.2)
Requirement already satisfied: diffusers in
/usr/local/lib/python3.12/dist-packages (0.37.1)
Requirement already satisfied: importlib-metadata in
/usr/local/lib/python3.12/dist-packages (from diffusers) (8.7.1)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from diffusers) (3.29.1)
Requirement already satisfied: httpx<1.0.0 in
/usr/local/lib/python3.12/dist-packages (from diffusers) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.34.0 in
/usr/local/lib/python3.12/dist-packages (from diffusers) (1.19.0)
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from diffusers) (2.0.2)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.12/dist-packages (from diffusers) (2025.11.3)
Requirement already satisfied: requests in
/usr/local/lib/python3.12/dist-packages (from diffusers) (2.32.4)
Requirement already satisfied: safetensors>=0.3.1 in
```

/usr/local/lib/python3.12/dist-packages (from diffusers) (0.7.0)
Requirement already satisfied: Pillow in
/usr/local/lib/python3.12/dist-packages (from diffusers) (11.3.0)
Requirement already satisfied: anyio in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->diffusers)
(4.13.0)
Requirement already satisfied: certifi in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->diffusers)
(2026.5.20)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0->diffusers)
(1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-
packages (from httpx<1.0.0->diffusers) (3.18)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.12/dist-packages (from httpcore==1.*-
>httpx<1.0.0->diffusers) (0.16.0)
Requirement already satisfied: click>=8.4.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (8.4.1)
Requirement already satisfied: fsspec>=2023.5.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (1.5.1)
Requirement already satisfied: packaging>=20.9 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (26.2)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (6.0.3)
Requirement already satisfied: tqdm>=4.42.1 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (4.67.3)
Requirement already satisfied: typer<0.26.0,>=0.20.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (0.25.1)
Requirement already satisfied: typing-extensions>=4.1.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface-
hub<2.0,>=0.34.0->diffusers) (4.15.0)
Requirement already satisfied: zipp>=3.20 in
/usr/local/lib/python3.12/dist-packages (from importlib-metadata-
>diffusers) (4.1.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests->diffusers)
(3.4.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests->diffusers)

(2.5.0)
Requirement already satisfied: shellingham>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub<2.0,>=0.34.0->diffusers) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in
/usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub<2.0,>=0.34.0->diffusers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0->huggingface-hub<2.0,>=0.34.0->diffusers) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface-hub<2.0,>=0.34.0->diffusers)
(4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface-hub<2.0,>=0.34.0->diffusers)
(2.20.0)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface-hub<2.0,>=0.34.0->diffusers) (0.1.2)
Requirement already satisfied: huggingface_hub in
/usr/local/lib/python3.12/dist-packages (1.19.0)
Requirement already satisfied: click>=8.4.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub) (8.4.1)
Requirement already satisfied: filelock>=3.10.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub)
(3.29.1)
Requirement already satisfied: fsspec>=2023.5.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub)
(2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub) (1.5.1)
Requirement already satisfied: httpx<1,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub)
(0.28.1)
Requirement already satisfied: packaging>=20.9 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub) (26.2)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub) (6.0.3)
Requirement already satisfied: tqdm>=4.42.1 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub)
(4.67.3)
Requirement already satisfied: typer<0.26.0,>=0.20.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub)
(0.25.1)
Requirement already satisfied: typing-extensions>=4.1.0 in
/usr/local/lib/python3.12/dist-packages (from huggingface_hub)

```

(4.15.0)
Requirement already satisfied: anyio in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface_hub) (4.13.0)
Requirement already satisfied: certifi in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface_hub) (2026.5.20)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
>huggingface_hub) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-
packages (from httpx<1,>=0.23.0->huggingface_hub) (3.18)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.12/dist-packages (from httpcore==1.*-
>httpx<1,>=0.23.0->huggingface_hub) (0.16.0)
Requirement already satisfied: shellingham>=1.3.0 in
/usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0-
>huggingface_hub) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in
/usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0-
>huggingface_hub) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from typer<0.26.0,>=0.20.0-
>huggingface_hub) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=13.8.0-
>typer<0.26.0,>=0.20.0->huggingface_hub) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=13.8.0-
>typer<0.26.0,>=0.20.0->huggingface_hub) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich>=13.8.0->typer<0.26.0,>=0.20.0->huggingface_hub) (0.1.2)

```

```

from huggingface_hub import HfApi, list_models

```

```

api = HfApi()

```

```

models = list(
    list_models(
        filter='text-generation',
        sort='downloads',
        limit=5
    )
)

```

```

print("Top 5 Text Generation Models:")

```

```

for m in models:
    print(

```

```

        f"{m.modelId} - {m.downloads:,} downloads"
    )

model_info = api.model_info(
    'ibm-granite/granite-3.1-2b-instruct'
)

print("\nModel Tags:")
print(model_info.tags)

Top 5 Text Generation Models:
Qwen/Qwen3-0.6B - 21,741,676 downloads
Qwen/Qwen3-4B - 15,850,097 downloads
openai-community/gpt2 - 13,182,834 downloads
Qwen/Qwen2.5-3B-Instruct - 13,113,728 downloads
Qwen/Qwen2.5-7B-Instruct - 11,898,561 downloads

Model Tags:
['transformers', 'safetensors', 'granite', 'text-generation',
 'language', 'granite-3.1', 'conversational', 'arxiv:0000.00000',
 'base_model:ibm-granite/granite-3.1-2b-base',
 'base_model:finetune:ibm-granite/granite-3.1-2b-base',
 'license:apache-2.0', 'eval-results', 'region:us']

from transformers import pipeline

fill_mask = pipeline(
    'fill-mask',
    model='bert-base-uncased'
)

prompts = [
    'The doctor finished his [MASK] work.',
    'The nurse finished her [MASK] work.',
    'The engineer completed the [MASK].',
    'The teacher helped [MASK] students.'
]

print("Bias Probe Results:")
print("="*60)

for prompt in prompts:
    results = fill_mask(prompt, top_k=3)

    print(f"\nPrompt: {prompt}")

    for r in results:
        print(
            f'[{r["score"]:.1%}] {r["token_str"]}'
        )

```

```
{"model_id": "2d19b6ead75a4178bed23a6a8aab6b67", "version_major": 2, "version_minor": 0}
```

[transformers] BertForMaskedLM LOAD REPORT from: bert-base-uncased

Key	Status		
cls.seq_relationship.bias	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
bert.pooler.dense.bias	UNEXPECTED		
bert.pooler.dense.weight	UNEXPECTED		

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Bias Probe Results:

=====

Prompt: The doctor finished his [MASK] work.

[14.1%] own
[4.9%] medical
[2.3%] dirty

Prompt: The nurse finished her [MASK] work.

[8.2%] own
[4.8%] dirty
[3.6%] exam

Prompt: The engineer completed the [MASK].

[8.3%] work
[4.0%] construction
[3.9%] project

Prompt: The teacher helped [MASK] students.

[71.2%] the
[5.0%] his
[2.6%] her

```
from huggingface_hub import InferenceClient
```

```
client = InferenceClient(  
    token="your_token"  
)
```

```
print("\n--- Image Generation ---")
```

```
try:
```

```
    image = client.text_to_image(  
        model=  
            "stabilityai/stable-diffusion-xl-base-1.0",
```

```

        prompt=
        "A photorealistic robot teaching students in front of a
classroom"
    )

    if image:
        image.save(
            "generated_image_client.jpg"
        )
        print(
            "Image saved successfully!"
        )
    else:
        print(
            "No image returned."
        )

except Exception as e:
    print(
        f"Error: {e}"
    )

--- Image Generation ---
Error: (Request ID: Root=1-6a2baff7-100dcb0a55fe54ed0fe7e38f;0cbf45b9-
3c38-484c-9914-818e560a16a0)

403 Forbidden: This authentication method does not have sufficient
permissions to call Inference Providers on behalf of user
sourabhvamdevan889.
Cannot access content at:
https://router.huggingface.co/nscale/v1/images/generations.
Make sure your token has the correct permissions.

```

Day 5 : Sentiment Analysis, Text Generation, and Generative Adversarial Networks (GANs)

```

from transformers import pipeline

classifier = pipeline('sentiment-analysis')

texts = [
    'Generative AI is revolutionizing every industry!',
    'AI systems are biased and dangerous.',
    'I am neutral about AI developments.'
]

results = classifier(texts)

for text, result in zip(texts, results):
    print(

```

```

        f'{{result["label"]:10}} '
        f'({result["score"]:.3f}) '
        f'| {{text[:60]}}'
    )

generator = pipeline(
    'text-generation',
    model='gpt2'
)

prompt = (
    'The future of generative AI in education is'
)

generated = generator(
    prompt,
    max_new_tokens=60,
    do_sample=True,
    temperature=0.9
)

print(
    f'\nGenerated text:\n'
    f'{{generated[0]["generated_text"]}}'
)

```

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.

```

{"model_id": "6f9cd6c0fe894f72bd9cdf9160d93df1", "version_major": 2, "version_minor": 0}

```

```

POSITIVE    (0.999) | Generative AI is revolutionizing every industry!
NEGATIVE    (1.000) | AI systems are biased and dangerous.
NEGATIVE    (0.990) | I am neutral about AI developments.

```

```

{"model_id": "70db26fc2246479d86bc78e7804c1242", "version_major": 2, "version_minor": 0}

```

[transformers] Passing `generation\_config` together with generation-related arguments=({'temperature', 'do_sample', 'max_new_tokens'}) is deprecated and will be removed in future versions. Please pass either a `generation\_config` object OR all generation parameters explicitly, but not both.

[transformers] Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

[transformers] Both `max\_new\_tokens` (=60) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence. Please refer

to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force cleanup anyway.

Generated text:

The future of generative AI in education is uncertain.

If we are serious about AI for a future, it is essential that there are more than a few of these technologies, not the number, that are available for teachers, to provide students with the knowledge they need to make informed decision-making decisions.

We can start by

Day 6 : Image Generation using Variational Autoencoder (VAE)

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import torch.optim as optim
import matplotlib.pyplot as plt

class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=20):
        super().__init__()

        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)

        self.fc3 = nn.Linear(latent_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, input_dim)

    def encode(self, x):
        h = F.relu(self.fc1(x))
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
```

```

        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h))

    def forward(self, x):
        mu, logvar = self.encode(x.view(-1, 784))
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

def vae_loss(recon_x, x, mu, logvar):

    BCE = F.binary_cross_entropy(
        recon_x,
        x.view(-1, 784),
        reduction='sum'
    )

    KLD = -0.5 * torch.sum(
        1 + logvar - mu.pow(2) - logvar.exp()
    )

    return BCE + KLD

transform = transforms.ToTensor()

dataset = torchvision.datasets.MNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

loader = DataLoader(
    dataset,
    batch_size=128,
    shuffle=True
)

print("Dataset Loaded Successfully!")

100%|██████████| 9.91M/9.91M [00:00<00:00, 40.9MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.15MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 9.53MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.08MB/s]

Dataset Loaded Successfully!

```

```

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using Device:", device)

vae = VAE().to(device)

optimizer = optim.Adam(
    vae.parameters(),
    lr=0.001
)

Using Device: cpu

epochs = 3

vae.train()

for epoch in range(epochs):
    total_loss = 0

    for images, _ in loader:
        images = images.to(device)

        recon_images, mu, logvar = vae(images)

        loss = vae_loss(
            recon_images,
            images,
            mu,
            logvar
        )

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        total_loss += loss.item()

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Loss: {total_loss:.2f}"
    )

```

```
Epoch [1/3] Loss: 9894450.59
Epoch [2/3] Loss: 7292570.23
Epoch [3/3] Loss: 6862517.41

vae.eval()

with torch.no_grad():
    z_sample = torch.randn(16, 20).to(device)
    generated = vae.decode(z_sample).cpu()

print("Generated shape:", generated.shape)
Generated shape: torch.Size([16, 784])

generated_images = generated.view(
    16,
    28,
    28
).numpy()

fig, axes = plt.subplots(
    4,
    4,
    figsize=(6, 6)
)

for i, ax in enumerate(axes.flat):
    ax.imshow(
        generated_images[i],
        cmap="gray"
    )

    ax.axis("off")

plt.show()
```



Day 7 : Prompt Engineering Techniques using Google Gemini

```
!pip install google-genai
```

```
Requirement already satisfied: google-genai in  
/usr/local/lib/python3.12/dist-packages (1.68.0)  
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in  
/usr/local/lib/python3.12/dist-packages (from google-genai) (4.13.0)  
Requirement already satisfied: google-auth<3.0.0,>=2.47.0 in  
/usr/local/lib/python3.12/dist-packages (from google-  
auth[requests]<3.0.0,>=2.47.0->google-genai) (2.47.0)  
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in  
/usr/local/lib/python3.12/dist-packages (from google-genai) (0.28.1)  
Requirement already satisfied: pydantic<3.0.0,>=2.9.0 in  
/usr/local/lib/python3.12/dist-packages (from google-genai) (2.12.3)  
Requirement already satisfied: requests<3.0.0,>=2.28.1 in  
/usr/local/lib/python3.12/dist-packages (from google-genai) (2.32.4)  
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in  
/usr/local/lib/python3.12/dist-packages (from google-genai) (9.1.4)  
Requirement already satisfied: websockets<17.0,>=13.0.0 in  
/usr/local/lib/python3.12/dist-packages (from google-genai) (15.0.1)  
Requirement already satisfied: typing-extensions<5.0.0,>=4.14.0 in
```

```

/usr/local/lib/python3.12/dist-packages (from google-genai) (4.15.0)
Requirement already satisfied: distro<2,>=1.7.0 in
/usr/local/lib/python3.12/dist-packages (from google-genai) (1.9.0)
Requirement already satisfied: sniffio in
/usr/local/lib/python3.12/dist-packages (from google-genai) (1.3.1)
Requirement already satisfied: idna>=2.8 in
/usr/local/lib/python3.12/dist-packages (from anyio<5.0.0,>=4.8.0-
>google-genai) (3.18)
Requirement already satisfied: pyasn1-modules>=0.2.1 in
/usr/local/lib/python3.12/dist-packages (from google-
auth<3.0.0,>=2.47.0->google-auth[requests]<3.0.0,>=2.47.0->google-
genai) (0.4.2)
Requirement already satisfied: rsa<5,>=3.1.4 in
/usr/local/lib/python3.12/dist-packages (from google-
auth<3.0.0,>=2.47.0->google-auth[requests]<3.0.0,>=2.47.0->google-
genai) (4.9.1)
Requirement already satisfied: certifi in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1-
>google-genai) (2026.5.20)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1-
>google-genai) (1.0.9)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.12/dist-packages (from httpcore==1.*-
>httpx<1.0.0,>=0.28.1->google-genai) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in
/usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.9.0-
>google-genai) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in
/usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.9.0-
>google-genai) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in
/usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.9.0-
>google-genai) (0.4.2)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1-
>google-genai) (3.4.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.28.1-
>google-genai) (2.5.0)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in
/usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1-
>google-auth<3.0.0,>=2.47.0->google-auth[requests]<3.0.0,>=2.47.0-
>google-genai) (0.6.3)

```

```

from google import genai
from google.colab import userdata

api_key = userdata.get('GOOGLE_API_KEY')

```

```

client = genai.Client(api_key=api_key)
print("Gemini Client Initialized Successfully")
Gemini Client Initialized Successfully

prompt = """
Classify the sentiment of the following review as Positive, Negative,
or Neutral.

Review: "The battery life on this new phone easily lasts two days, but
the screen is a bit dimmer than I expected."

Sentiment:
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("--- Zero-Shot Prompting ---")
print(response.text)

--- Zero-Shot Prompting ---
Neutral

prompt = """
Convert the following movie description into a list of 3 relevant
emojis.

Example:

Input: A group of astronauts travel through a wormhole in space to
ensure humanity's survival.

Output:

Task:

Input: A young boy discovers a magical ring and goes on an epic quest
to destroy it in a volcano.

Output:
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

```

```

print("--- One-Shot Prompting ---")
print(response.text)

--- One-Shot Prompting ---
[]

prompt = """
Extract the product and the specific complaint from customer emails.

Example 1:

Input: "Hey, I bought your NoiseCancel Headphones yesterday, but the
Bluetooth keeps disconnecting every 5 minutes!"

Output: Product: NoiseCancel Headphones | Complaint: Intermittent
Bluetooth disconnection.

Example 2:

Input: "The SmartBlender 3000 arrived today. It blends well, but the
plastic lid was completely cracked in the box."

Output: Product: SmartBlender 3000 | Complaint: Damaged parts upon
delivery.

Task:

Input: "My AquaStream Water Filter is leaking from the bottom valve
every time I try to purify water."

Output:
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("--- Few-Shot Prompting ---")
print(response.text)

--- Few-Shot Prompting ---
Product: AquaStream Water Filter | Complaint: Leakage from bottom
valve during use.

prompt = """
Solve the following math word problem.

Think step-by-step before giving the final answer.

Problem:

```

A cafeteria has 23 apples.
If they use 20 to make lunch and buy 6 more,
how many apples do they have now?

Reasoning:

"""

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents=prompt  
)
```

```
print("--- Chain of Thought Prompting ---")  
print(response.text)
```

--- Chain of Thought Prompting ---
Here's a step-by-step solution:

1. ****Start with the initial number of apples:**** The cafeteria has 23 apples.
2. ****Subtract the apples used for lunch:**** They use 20 apples.
 $23 - 20 = 3$ apples
3. ****Add the apples bought:**** They buy 6 more apples.
 $3 + 6 = 9$ apples

The cafeteria has 9 apples now.

prompt = """

Imagine three different expert logistics strategists are solving this problem.

They will think step-by-step, critique each other's ideas, and build upon the best path.

Problem:

A company needs to ship 500 fragile glass vases from New York to Los Angeles.

They need to minimize cost, but if more than 5% of vases break, they lose their profit margin.

Options:

1. Fast air freight
2. Standard ground trucking
3. Dedicated padded transport

Generate the discussion between the experts to find the optimal strategy.

```
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("--- Tree of Thoughts Prompting ---")
print(response.text)
```

--- Tree of Thoughts Prompting ---

The conference room buzzed with the low hum of a projector. Three seasoned logistics strategists, Aisha, Ben, and Chloe, gathered to tackle the delicate problem of shipping 500 fragile glass vases.

Aisha (The Pragmatist): "Alright team, 500 fragile glass vases, NY to LA. Minimize cost, but absolutely no more than 5% breakage, or we lose our entire profit margin. That's a critical constraint. We have three main options on the table: Fast air freight, standard ground trucking, or dedicated padded transport."

Ben (The Risk Averse): "My immediate concern is the 'fragile glass vases' and that '5% breakage' threshold. Standard ground trucking immediately makes me nervous. The vibrations, multiple handling points, potential for shifting cargo in a mixed load – it's a high-risk gamble for fragile items, even with good packaging. We're talking about losing our entire profit, potentially a five-figure sum, if we exceed 25 broken vases."

Chloe (The Optimiser): "I agree with Ben on the inherent risks of standard ground. However, we can't dismiss it outright without understanding *why* it's 'standard' and what modifications are possible. But let's start with a clearer picture of each. Aisha, what are the ballpark cost implications and typical transit times we're looking at for each option?"

Aisha: "Good question, Chloe. Let's ballpark some figures for discussion:

- Fast air freight:** Likely the fastest, say 1-2 days transit. Cost could be in the range of \$8,000 - \$12,000. Breakage risk, while lower than standard ground, still exists due to cargo handling and pressure changes, especially if not packed impeccably. Let's estimate 2-3% breakage even with decent packaging, but potentially higher if rushed.
- Standard ground trucking (LTL - Less Than Truckload):** The cheapest transport, maybe \$2,500 - \$4,000, with transit around 5-7 days. This is where Ben's concerns are most valid. Without significant intervention, I'd estimate a 10-15% breakage rate, easily exceeding our threshold.

3. ****Dedicated padded transport:**** This would mean a specialized truck, often air-ride suspension, possibly climate-controlled, and with minimal handling. Transit 3-5 days. This is likely the most expensive, possibly \$15,000 - \$25,000, as we're essentially paying for a full truck for only 500 vases. Breakage would be minimal, likely under 1%."

****Ben:**** "Okay, so dedicated padded transport **guarantees** we meet our breakage target, but at potentially 5-10 times the cost of standard ground. Air freight is fast, and safer than ground, but still expensive and not foolproof. Standard ground, at \$2,500-\$4,000, sounds attractive **until** you factor in an estimated 10-15% breakage. If 10% break, that's 50 vases. If each vase costs, say, \$50 to produce, that's \$2,500 in lost product **plus** the entire profit margin. So, a \$4,000 shipping cost suddenly becomes \$4,000 + \$2,500 + 'profit margin' (let's assume \$10,000 for calculation) = \$16,500. That makes it more expensive than even some air freight options."

****Chloe:**** "Exactly, Ben. The 'true cost' isn't just the shipping invoice; it's shipping plus the **expected cost of breakage**. That immediately rules out standard, unmitigated LTL ground trucking. It's a false economy. However, option 2 isn't just 'standard ground' or 'dedicated padded.' There's a spectrum in between. What if we invest heavily in **packaging**? We're talking custom foam inserts, double-walled corrugated boxes, palletizing, maybe even crate-and-cushioning for entire pallets. This shifts the risk profile dramatically for ground transport."

****Aisha:**** "That's a very valid point, Chloe. Superior packaging is often overlooked as part of the transport strategy itself. Let's explore that."

* For ****Air Freight****, investing in premium packaging could bring that 2-3% breakage down to under 1%, virtually eliminating the risk. But it adds to the already high cost.

* For ****Standard Ground (LTL)****, if we used extremely robust, custom packaging, specifically designed to withstand vibrations and minor impacts, and marked the pallets clearly 'FRAGILE - DO NOT STACK - TOP LOAD ONLY,' we could significantly reduce that 10-15% breakage. It still wouldn't be as good as dedicated, but perhaps below the 5% threshold."

****Ben:**** "How much would that premium packaging cost, though? If it adds another \$2,000-\$3,000 to the ground freight cost, pushing it to \$6,000-\$7,000, and still only **hopes** to get us under 5% breakage, is it worth the risk? The entire profit margin is on the line. I'd rather spend a bit more upfront to practically guarantee we don't cross that 5% line."

****Chloe:**** "I appreciate your cautiousness, Ben. But let's break it down. For 500 vases, if custom packaging costs, say, \$5-\$10 per vase,

that's \$2,500-\$5,000.

* **Option 2 Revised (LTL Ground + Premium Packaging):** Base transport (\$3,500 average) + Packaging (\$4,000 average) = **\$7,500 total**. Expected breakage, with optimal packaging and careful carrier selection for LTL fragile goods, could be brought down to 3-4%. This keeps us under the 5% threshold, and the breakage cost (e.g., 4% of \$50 = \$2 per vase * 500 = \$1,000 lost product value) is absorbed without losing profit margin. Total risk adjusted cost: ~\$7,500 + \$1,000 = **\$8,500.**

* **Option 1 Revised (Air Freight + Premium Packaging):** Base transport (\$10,000 average) + Packaging (\$4,000 average) = **\$14,000 total**. Expected breakage perhaps 0.5-1%. Total risk adjusted cost: ~\$14,000 + \$250 = **\$14,250.**

* **Option 3 (Dedicated Padded Transport):** Base transport (\$20,000 average). Expected breakage under 1%. Total risk adjusted cost: ~\$20,000 + \$250 = **\$20,250.**"

Aisha: "Chloe's breakdown makes the picture much clearer. The 'sweet spot' appears to be the optimized LTL ground solution. It's significantly cheaper than air freight and dramatically cheaper than dedicated, while still mitigating the risk to an acceptable level. The key really is that investment in packaging and being explicit with the LTL carrier about the fragility and handling requirements. We need to select an LTL carrier known for careful handling and offer specific instructions, perhaps even a premium service for fragile goods."

Ben: "I'm still a little uneasy relying solely on 'careful carrier selection' and 'explicit instructions' for LTL. These are often high-volume operations where individual package care can be inconsistent. However, with *exceptional* packaging – packaging so robust that the vase itself is almost impervious to typical LTL bumps and shifts – it becomes more palatable. We would need to specify air-ride trailers if possible for LTL, and ensure it stays on its original pallet without being trans-loaded multiple times at different hubs, which some specialized LTL carriers can offer."

Chloe: "Precisely, Ben. It's not just 'packaging'; it's 'engineering the packaging to survive a typical LTL journey.' We could also look at a 'partial truckload' or 'volume LTL' service. This is often more expensive than standard LTL but less than a dedicated truck. It offers fewer touchpoints and more control over the space the shipment occupies, further reducing risk without the full dedicated truck price tag. This acts as a middle ground between standard LTL and full dedicated."

Aisha: "That's an excellent refinement, Chloe. A 'partial truckload' or 'guaranteed service LTL' with an air-ride trailer and our engineered packaging would provide a robust solution. Let's revise the 'Option 2 Revised' cost for that slightly higher service level. Maybe \$5,000-\$6,000 for transport, plus the \$4,000 for packaging,

bringing us to around ****\$9,000-\$10,000 total operational cost****. The breakage rate for this highly-mitigated ground option should be comfortably under 3%, which is well within our 5% tolerance. This strategy provides excellent value for money."

****Ben:**** "Given those numbers and the layered mitigation strategy – engineered packaging, air-ride LTL, fewer touchpoints – I'm significantly more comfortable with the ground option. We're talking about saving roughly \$4,000-\$5,000 compared to air freight, with comparable risk, and avoiding the astronomical cost of a dedicated truck. The crucial part is vetting the carrier and ensuring they understand the 'fragile' nature despite the robust packaging."

****Aisha:**** "Agreed. The final strategy would be:

1. ****Invest in Superior, Engineered Packaging:**** Custom foam inserts, double-walled boxes, shrink-wrapped and corner-protected on sturdy pallets. This is non-negotiable for any option. Estimated cost: \$4,000-\$5,000.
2. ****Select a Specialized LTL Carrier (Partial Truckload/Guaranteed Service):**** Look for carriers offering 'fragile goods' service, air-ride trailers, and minimal trans-load points for long-haul. Explicitly communicate the fragility and 'no-stack' instructions. Estimated transport cost: \$5,000-\$6,000.
3. ****Total Estimated Cost (Operational):**** \$9,000-\$11,000.
4. ****Expected Breakage:**** Under 3%, well within the 5% threshold, protecting our profit margin.

This provides the best balance of cost-efficiency and risk mitigation."

****Chloe:**** "Consensus reached. This optimized ground strategy leverages cost savings where possible, but critically invests in the areas that directly address the core problem of fragility and the 5% threshold. It's not just picking an option; it's **designing** the optimal shipping process."

****Ben:**** "Excellent. My risk-averse nature is appeased, and my wallet feels happier too. Let's make sure the packaging specs are robust enough for this to truly work."

```
from google.genai import types
```

```
config = types.GenerateContentConfig(  
    system_instruction=  
        "You are a strict, no-nonsense Cyber Security Auditor. You speak  
        bluntly and prioritize absolute security."  
)
```

```
prompt = ""
```

Our team wants to store user passwords in plain text temporarily in our database just while we test a feature.

Is that okay?

"""

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents=prompt,  
    config=config  
)
```

```
print("--- Persona-Based Prompting ---")  
print(response.text)
```

--- Persona-Based Prompting ---

No. Absolutely not. This is a non-negotiable security violation.

Storing user passwords in plain text, *even temporarily*, for *any* reason, introduces a catastrophic risk of data breach. Do you comprehend the implications? A single compromise—a leaked backup, an accidental log, an unauthorized query—and every one of your users' accounts is exposed. Not just for your system, but potentially across every other service where they've reused that password.

This isn't a shortcut; it's a gaping security hole you are deliberately creating.

Passwords must *always* be hashed with a strong, modern, salt-aware algorithm (e.g., Argon2, scrypt, bcrypt) from the moment they are received. This applies to *all* environments: production, staging, development, and testing. There are zero exceptions to this rule.

If your "feature" requires plain text passwords, then your feature is fundamentally flawed and needs to be re-architected. Immediately.

Do not proceed with this plan. It will be flagged as a critical security failure, and there will be severe repercussions.

```
from google.genai import types
```

```
config = types.GenerateContentConfig(  
    system_instruction=  
        "You are a strict, no-nonsense Cyber Security Auditor. You speak bluntly and prioritize absolute security."  
)
```

```
prompt = """
```

```
Our team wants to store user passwords in plain text temporarily in  
our database just while we test a feature.
```

```
Is that okay?  
"""
```

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents=prompt,  
    config=config  
)
```

```
print("--- Persona-Based Prompting ---")  
print(response.text)
```

```
--- Persona-Based Prompting ---  
Absolutely not. Are you serious?
```

Storing user passwords in plain text, *even temporarily*, is a catastrophic security failure waiting to happen. It is a fundamental, non-negotiable violation of every basic security principle.

The word "temporary" means absolutely nothing when a database can be compromised in seconds. If that data is breached, you've just handed over every user's credentials on a silver platter.

This is not "okay." This is *never* okay. Hash and salt your passwords, always. End of discussion. Find another way to test.

```
response = chat.send_message(  
    "I have experience with Python and Machine Learning."  
)
```

```
print(response.text)
```

That's great to hear! Python and Machine Learning are highly sought-after skills.

To start, could you tell me about a project where you applied your Python and Machine Learning skills to solve a real-world problem? I'd be interested to hear about the problem itself, your approach, and the impact of your solution.

Day 8 Retrieval-Augmented Generation (RAG) and Self-Consistency Prompting using FLAN-T5

```
!pip -q install transformers torch sentencepiece accelerate
```

```
from transformers import AutoTokenizer  
from transformers import AutoModelForSeq2SeqLM  
  
tokenizer = AutoTokenizer.from_pretrained(  
    "google/flan-t5-large"  
)
```

```

model = AutoModelForSeq2SeqLM.from_pretrained(
    "google/flan-t5-large"
)

def generate(prompt):
    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    ).input_ids

    outputs = model.generate(
        inputs,
        max_new_tokens=512
    )

    return tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    )

print("FLAN-T5 Loaded Successfully")

{"model_id": "d6e8b2e9d97f4a0cb7a9f7d1beb15d9c", "version_major": 2, "version_minor": 0}

{"model_id": "47ea56dc1b0d4e428d70c6edf45e0546", "version_major": 2, "version_minor": 0}

{"model_id": "9f51e58f13d041c391b4b067a17e4fba", "version_major": 2, "version_minor": 0}

{"model_id": "549531c7ef4d4ab08b7fd8134d3874df", "version_major": 2, "version_minor": 0}

{"model_id": "84c1dab9d9134e89b9671c08e284dda9", "version_major": 2, "version_minor": 0}

{"model_id": "e5664dd6a64942fba99d6f0789531218", "version_major": 2, "version_minor": 0}

{"model_id": "eb7d49ae9ee649e98e55efed5ee3f612", "version_major": 2, "version_minor": 0}

[transformers] The tied weights mapping and config for this model
specifies to tie shared.weight to lm_head.weight, but both are present
in the checkpoints with different values, so we will NOT tie them. You
should update the config with `tie_word_embeddings=False` to silence
this warning.

{"model_id": "3ad38f72e4394952ac2fadb9dc43260f", "version_major": 2, "version_minor": 0}

```

FLAN-T5 Loaded Successfully

```
documents = ""
```

```
Document 1:
```

```
Node.js uses Google's V8 engine.
```

```
Document 2:
```

```
Express.js is a Node.js framework.
```

```
Document 3:
```

```
MongoDB is a NoSQL database.
```

```
""
```

```
question = (
```

```
    "Which framework is commonly used with Node.js?"
```

```
)
```

```
prompt = f"""
```

```
You are a company knowledge assistant.
```

```
CONTEXT:
```

```
{documents}
```

```
RULES:
```

```
1. Answer ONLY from context.
```

```
2. If answer not found say:
```

```
Information not available in provided context.
```

```
3. Mention source document.
```

```
QUESTION:
```

```
{question}
```

```
""
```

```
print(generate(prompt))
```

```
Express.js
```

```
from collections import Counter
```

```
import re
```

```
question = ""
```

```
A company sells a product for ₹500.
```

```
Manufacturing Cost = ₹300
```

```
Marketing Cost = ₹50
```

```
Tax = ₹25
```

Calculate the profit per product using the following steps:

1. Calculate Total Cost

Total Cost = Manufacturing Cost + Marketing Cost + Tax

2. Calculate Profit

Profit = Selling Price - Total Cost

What is the profit per product?

Think step by step and give final answer as:

FINAL ANSWER: <value>
"""

```
results = []  
  
for i in range(5):  
    response = generate(question)  
  
    print(f"\n--- Run {i+1} ---")  
    print(response)  
  
    match = re.search(  
        r'(?:(FINAL ANSWER|The answer)[: ]*(.*)',  
        response,  
        re.IGNORECASE  
    )  
  
    if match:  
        results.append(  
            match.group(1).strip()  
        )  
  
vote = Counter(results)  
  
print("\n=====")  
print("Majority Decision")  
print("=====")  
  
if vote:  
    print(vote.most_common(1)[0])  
else:  
    print(  
        "No valid answers were extracted."  
    )
```

```

--- Run 1 ---
Total cost = 300 + 50 + 25 = 500. Selling price = 500 - 500 = 250.
Profit = 250 - 500 = 250. The final answer: 250.

--- Run 2 ---
Total cost = 300 + 50 + 25 = 500. Selling price = 500 - 500 = 250.
Profit = 250 - 500 = 250. The final answer: 250.

--- Run 3 ---
Total cost = 300 + 50 + 25 = 500. Selling price = 500 - 500 = 250.
Profit = 250 - 500 = 250. The final answer: 250.

--- Run 4 ---
Total cost = 300 + 50 + 25 = 500. Selling price = 500 - 500 = 250.
Profit = 250 - 500 = 250. The final answer: 250.

--- Run 5 ---
Total cost = 300 + 50 + 25 = 500. Selling price = 500 - 500 = 250.
Profit = 250 - 500 = 250. The final answer: 250.

=====
Majority Decision
=====
('250.', 5)

```

Day 9: Generation and Evaluation of Synthetic Data using Conditional Tabular GAN (CTGAN)

```

!pip install ctgan --quiet
!pip install table-evaluator --no-dependencies --quiet
!pip install visual_evaluation --quiet 2>/dev/null || true

===== 0.0/74.5 kB ? eta -:--:--
===== 74.5/74.5 kB 3.7 MB/s eta
0:00:00
===== 2.0/2.0 MB 34.4 MB/s eta
0:00:00
===== 82.2/82.2 kB 3.0 MB/s eta
0:00:00

import pandas as pd
import warnings

from sklearn.datasets import (
    fetch_california_housing
)

from ctgan import CTGAN

from table_evaluator import (

```

```

    TableEvaluator
)
warnings.filterwarnings('ignore')
print("--- Step 1: Loading Real Dataset ---")

housing = fetch_california_housing(
    as_frame=True
)

real_data = housing.frame

print(
    f"Real dataset shape: "
    f"{real_data.shape}"
)

print(
    f"Columns: "
    f"{list(real_data.columns)}"
)

--- Step 1: Loading Real Dataset ---
Real dataset shape: (20640, 9)
Columns: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
'AveOccup', 'Latitude', 'Longitude', 'MedHouseVal']

print("--- Step 2: Training CTGAN ---")

discrete_columns = []

model = CTGAN(
    epochs=5,
    verbose=True
)

model.fit(
    real_data,
    discrete_columns
)

print(
    "\nCTGAN Training Completed!"
)

--- Step 2: Training CTGAN ---
Gen. (-01.34) | Discrim. (-00.05): 100%| ██████████ | 5/5 [00:13<00:00,
2.69s/it]

```

CTGAN Training Completed!

```
print(
    "\nGenerating synthetic data..."
)

synthetic_data = model.sample(500)

print(
    f"Synthetic dataset shape: "
    f"{synthetic_data.shape}"
)

Generating synthetic data...
Synthetic dataset shape: (500, 9)

print(
    "--- Real vs Synthetic Data Preview ---"
)

pd.set_option(
    'display.max_columns',
    4
)

pd.set_option(
    'display.width',
    1000
)

print(
    "\n[REAL DATA SAMPLE]"
)

print(
    real_data.head().round(3)
)

print("\n" + "="*50)

print(
    "\n[SYNTHETIC DATA SAMPLE]"
)

print(
    synthetic_data.head().round(3)
)
```

```

print("="*50)

--- Real vs Synthetic Data Preview ---

[REAL DATA SAMPLE]
  MedInc  HouseAge  ...  Longitude  MedHouseVal
0   8.325    41.0   ...   -122.23      4.526
1   8.301    21.0   ...   -122.22      3.585
2   7.257    52.0   ...   -122.24      3.521
3   5.643    52.0   ...   -122.25      3.413
4   3.846    52.0   ...   -122.25      3.422

[5 rows x 9 columns]

=====

[SYNTHETIC DATA SAMPLE]
  MedInc  HouseAge  ...  Longitude  MedHouseVal
0   6.395    9.193   ...   -117.912      1.959
1   5.243   41.276   ...   -122.548      1.745
2   3.691   18.867   ...   -120.686      1.601
3   3.362   24.587   ...   -121.598      5.103
4   4.867   20.101   ...   -122.285      3.885

[5 rows x 9 columns]

=====

print(
    "--- Step 3: Evaluating Quality ---"
)

real_sample = real_data.sample(
    500,
    random_state=42
)

evaluator = TableEvaluator(
    real_sample,
    synthetic_data
)

print(
    "Generating Distribution Plots..."
)

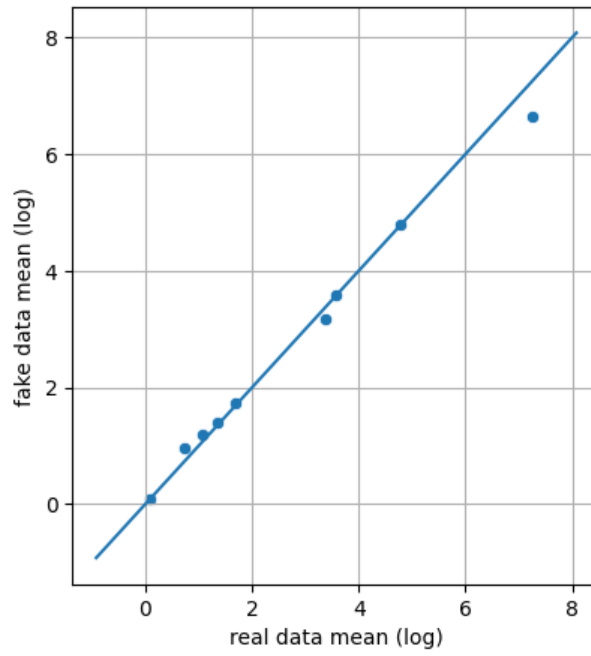
evaluator.visual_evaluation()

--- Step 3: Evaluating Quality ---
Generating Distribution Plots...

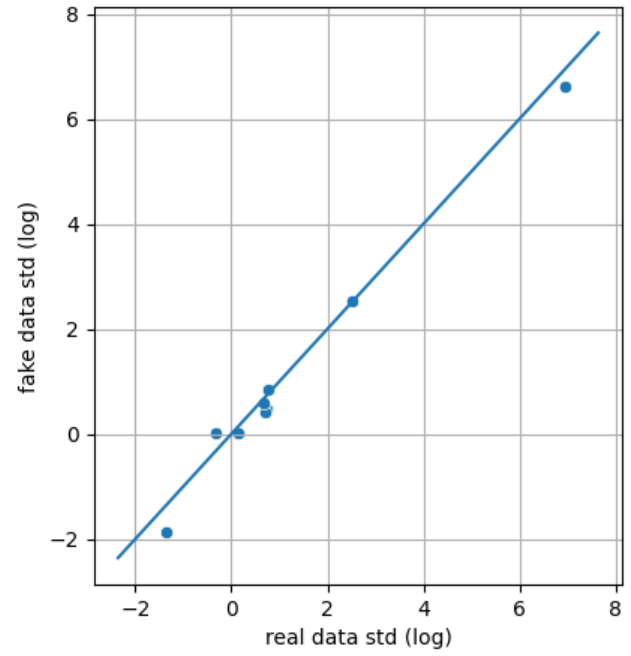
```

Absolute Log Mean and STDs of numeric data

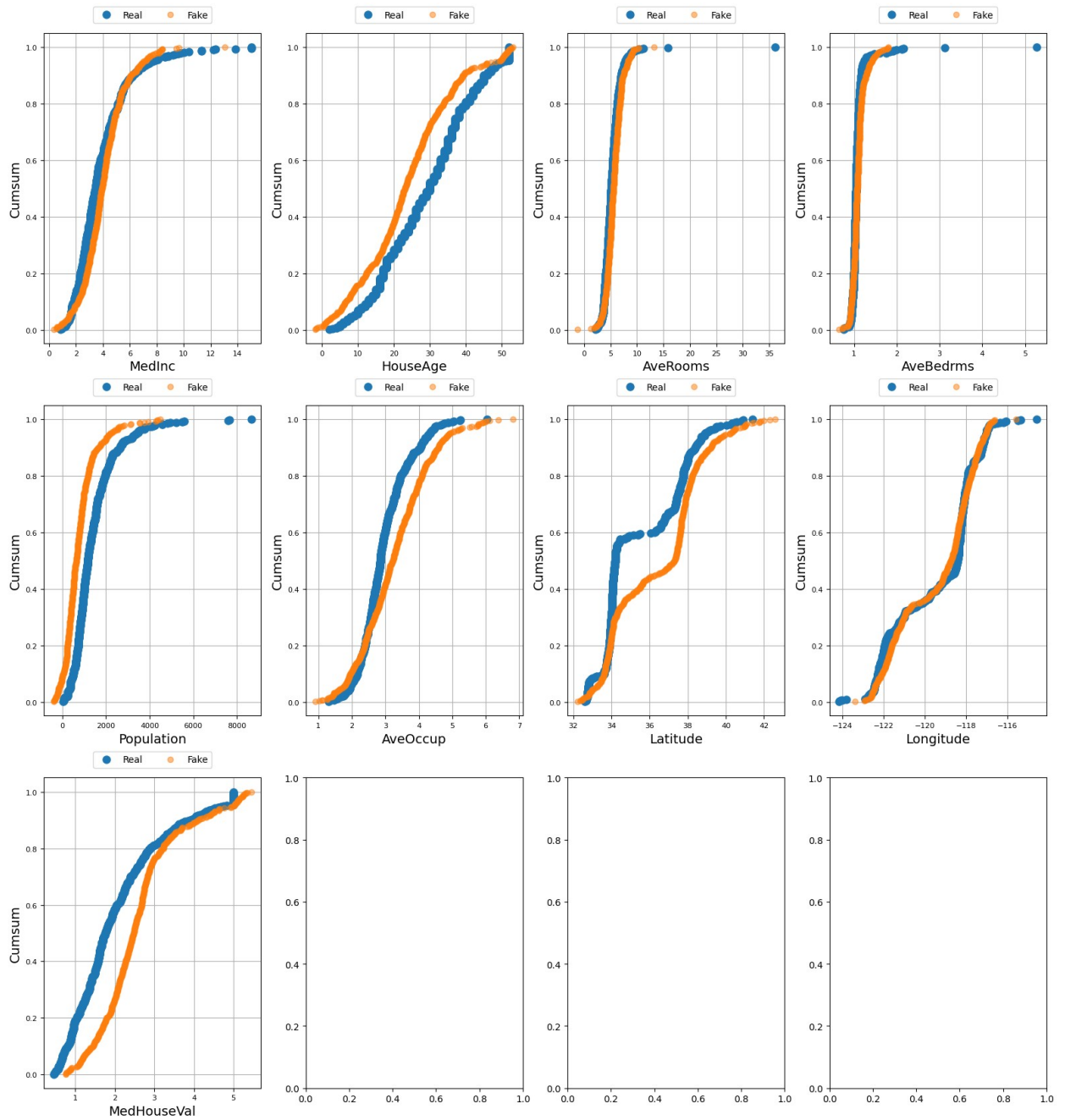
Means of real and fake data



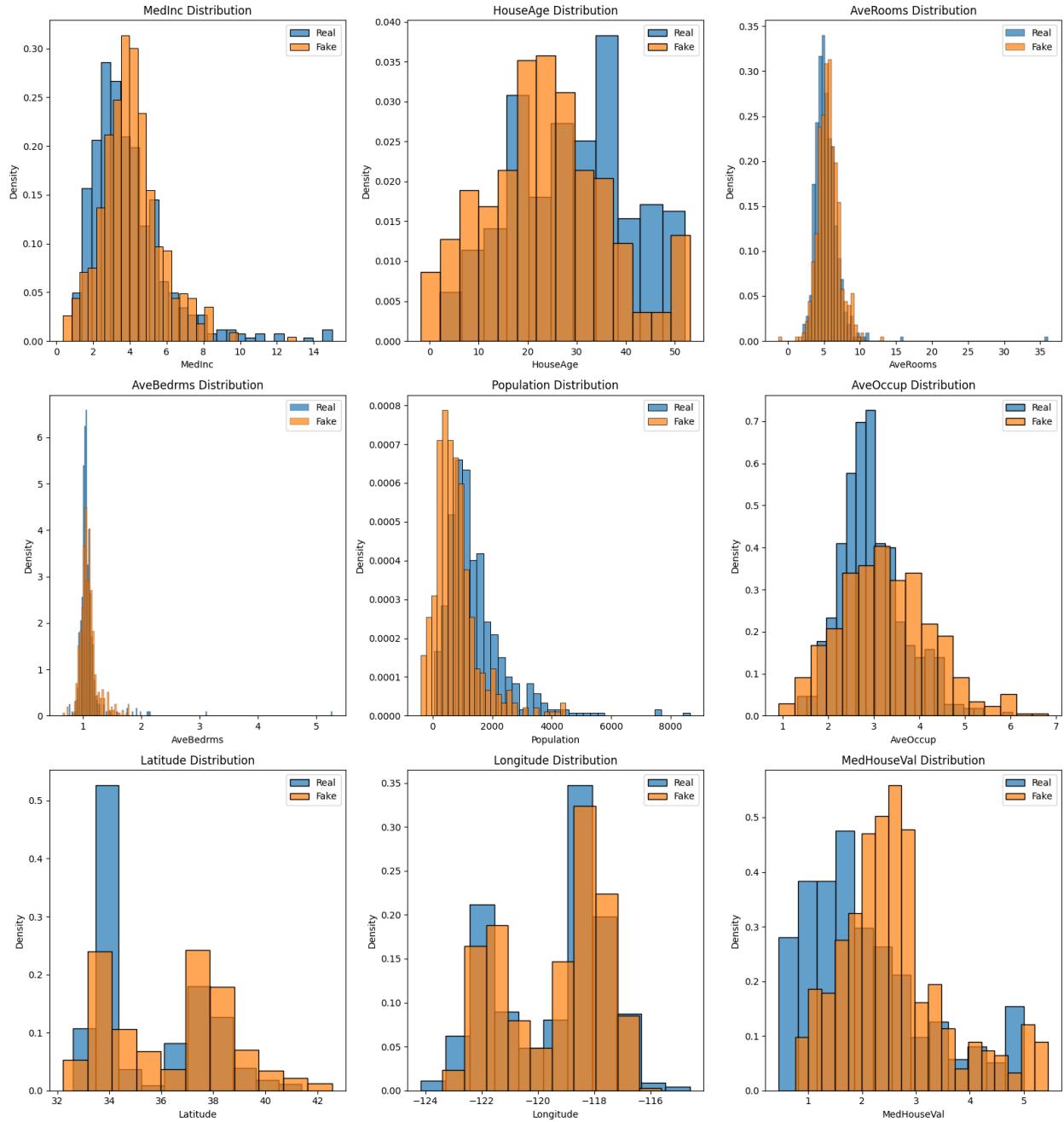
Stds of real and fake data

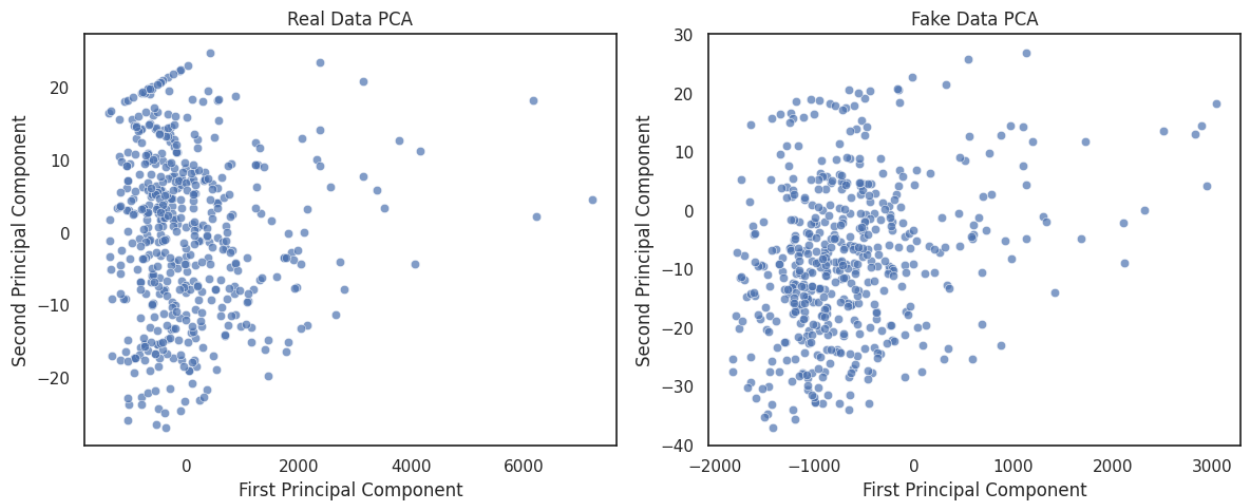
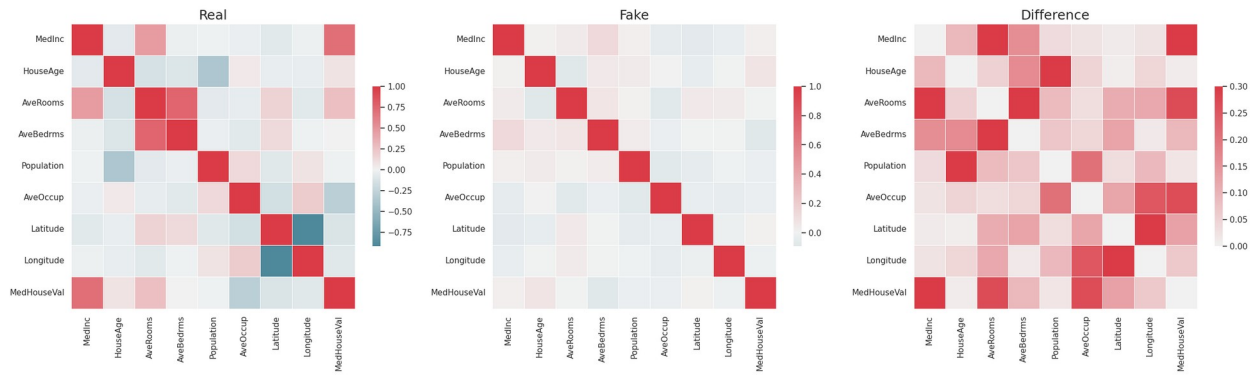


Cumulative Sums per feature



Distribution per feature





```
print(
    "\nCalculating Similarity Scores..."
)

evaluator.evaluate(
    target_col='MedHouseVal',
    target_type='regr'
)
```

Calculating Similarity Scores...

Regressor MSE-scores:

	real	fake
0	-0.8459	-0.8459

Privacy results:

	result
Duplicate rows between sets (real/fake)	(0, 0)
nearest neighbor mean	1.5332
nearest neighbor std	0.9551

Miscellaneous results:

	Result
Column Correlation Distance RMSE	0.2412
Column Correlation distance MAE	0.1471

Results:

	result
Basic statistics	0.9732
Correlation column correlations	0.2516
Mean Correlation between fake and real columns	0.9331
Correlation RMSE	-0.8459
Similarity Score	0.3280

```
{'Basic statistics': np.float64(0.9732303732303732),
 'Correlation column correlations': np.float64(0.2515952848867171),
 'Mean Correlation between fake and real columns':
 np.float64(0.933108469074139),
 'Correlation RMSE': np.float64(-0.8459127059347571),
 'Similarity Score': np.float64(0.328005355314118)}
```

Day 10 : Building a Data Pipeline for Retail Analytics using Python and SQLite

```
!pip install -q pandas numpy openpyxl
```

```
import pandas as pd
import numpy as np
import requests
import zipfile
import io
import sqlite3
import json
import os

from datetime import datetime
from pathlib import Path

print("Ready!")

Ready!

for folder in [
    "/content/data/raw/",
    "/content/data/processed/",
    "/content/data/gold/"
]:
    os.makedirs(
        folder,
        exist_ok=True
    )

URL = (
```

```

    "https://archive.ics.uci.edu/static/public/352/online+retail.zip"
)
print("Configuration set")
Configuration set
print("="*50)
print("STEP 1: Downloading Data")
print("="*50)

response = requests.get(
    URL,
    timeout=30
)

with zipfile.ZipFile(
    io.BytesIO(response.content)
) as z:

    excel_file = [
        f for f in z.namelist()
        if f.endswith('.xlsx')
    ][0]

    with z.open(excel_file) as f:

        df = pd.read_excel(
            f,
            engine='openpyxl'
        )

=====
STEP 1: Downloading Data
=====

df['CustomerID'] = (
    df['CustomerID']
    .fillna('UNKNOWN')
    .astype(str)
)

df['InvoiceNo'] = (
    df['InvoiceNo']
    .astype(str)
)

df['StockCode'] = (
    df['StockCode']
    .astype(str)
)

```

```

df['Description'] = (
    df['Description']
    .fillna('Unknown')
    .astype(str)
)

df['InvoiceDate'] = pd.to_datetime(
    df['InvoiceDate'],
    format='%m/%d/%Y %H:%M',
    errors='coerce'
)

df['Country'] = (
    df['Country']
    .astype(str)
)

print(
    f"Downloaded {len(df):,} rows"
)

print(
    f"Shape: {df.shape}"
)

df.head(3)

Downloaded 541,909 rows
Shape: (541909, 8)

{"type": "dataframe", "variable_name": "df"}

print("="*50)
print("STEP 2: Saving Raw Data")
print("="*50)

df.to_csv(
    "/content/data/raw/raw_data.csv",
    index=False
)

print(
    "Saved: raw_data.csv"
)

```

```

=====
STEP 2: Saving Raw Data
=====

```

```

Saved: raw_data.csv

```

```

conn = sqlite3.connect(
    "/content/data/gold/retail_warehouse.db"
)

df.to_sql(
    'transactions',
    conn,
    if_exists='replace',
    index=False
)

conn.close()

print(
    "Saved to database"
)

Saved to database

df.to_json(
    "/content/data/raw/raw_data.json",
    orient='records',
    lines=True
)

print("Saved as JSON")

print(
    "\nFiles saved in /content/data/raw/"
)

Saved as JSON

Files saved in /content/data/raw/

print("="*50)
print("STEP 3: Adding Derived Columns")
print("="*50)

df['TotalAmount'] = (
    abs(df['Quantity'])
    * df['UnitPrice']
)

df['Year'] = (
    df['InvoiceDate']
    .dt.year
    .fillna(2010)
    .astype(int)
)

```

```

df['Month'] = (
    df['InvoiceDate']
    .dt.month
    .fillna(1)
    .astype(int)
)

df['Hour'] = (
    df['InvoiceDate']
    .dt.hour
    .fillna(0)
    .astype(int)
)

df['DayOfWeek'] = (
    df['InvoiceDate']
    .dt.dayofweek
    .fillna(0)
    .astype(int)
)

df['Weekend'] = (
    df['DayOfWeek'] >= 5
).astype(int)

```

STEP 3: Adding Derived Columns

```

df['TransactionType'] = (
    df['InvoiceNo']
    .apply(
        lambda x:
            'CANCELLATION'
            if x.startswith('C')
            else 'SALE'
    )
)

def get_category(code):
    first = str(code)[0] if str(code) else 'X'

    return {
        '2': 'Electronics',
        '3': 'Home',
        '8': 'Gifts',
        '9': 'Seasonal'
    }.get(first, 'Other')

df['ProductCategory'] = (

```

```

    df['StockCode']
    .apply(get_category)
)

print(
    "Derived columns created successfully"
)

```

Derived columns created successfully

```

print(
    f"Shape: {df.shape}"
)

```

```

display(
    df[
        [
            'InvoiceNo',
            'TotalAmount',
            'TransactionType',
            'ProductCategory'
        ]
    ].head(5)
)

```

Shape: (541909, 16)

```

{"summary":{"\n  \"name\": \"\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"InvoiceNo\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"536365\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"TotalAmount\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 2.543242025447048,\n        \"min\": 15.299999999999999,\n        \"max\": 22.0,\n        \"num_unique_values\": 3,\n        \"samples\": [\n          15.299999999999999\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"TransactionType\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"SALE\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"ProductCategory\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"Other\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}, \"type\": \"dataframe\"}

import os

```

```
for f in os.listdir():
    print(f)

.config
data
sample_data

!ls

data sample_data

!find /content -name "*.ipynb"

import os

print(os.getcwd())
print("\nFiles:")
for f in os.listdir():
    print(f)

/content

Files:
.config
data
sample_data

!find /content -name "*.ipynb"
```